

Repetition

CSCI 1101

2024-02-29

Repetition

CSCI 1101

2024-02-29

Repeating Things

Sometimes we want to do something repeatedly, but we don't know how many times.

We may want to do it *until* some condition is met, or alternatively, *while* some condition is true.

For this, python has an iteration statement called a **while loop**.

It is written as follows:

```
while <guard> :  
    <loop_body>
```

where the **guard** is any boolean expression and the **loop body** is any nonempty sequence of statements (i.e. a **block**).

Loop Execution

When python encounters a `while` loop the following happens:

- The guard is evaluated to a boolean value (i.e. “tested”).
- If the guard evaluates to `False` then the loop body is *skipped* and control passes to the next statement in the program after the loop.
- If the guard evaluates to `True` then the loop body is *run*:
 - control passes to beginning of the loop body,
 - control flows until the end of the loop body,
 - after the last statement of the loop body, the entire process is repeated, beginning with the guard being tested again.

Example: Flipping a Coin

With the help of the `random` module, we can flip a coin:

```
import random

def flip() :
    return random.choice(["heads", "tails"])
```

We can keep flipping *until* we get heads:

```
def flip_until_heads() :
    coin = flip()
    print(coin)
    while coin != "heads" :
        coin = flip()
        print(coin)
```

Doing Something at Least Once

This illustrates a common pattern: doing something at least one time.

Instead of doing something just before the loop and then again inside of the loop, we can “engineer” the guard to succeed the first time.

```
def flip_until_heads() :  
    coin = "" # carefully chosen initial value  
    while coin != "heads" :  
        coin = flip()  
        print(coin)
```

It is important that we don't rely on the bogus initial value for anything except making the first iteration of the loop.

Activity: Flipping two Coins

Write a function `flip_until_same` that flips two coins repeatedly until they come up the same.

For example:

```
> flip_until_same()  
tails , heads  
heads , tails  
tails , tails
```

Hint: try using tuple assignment.

Activity: a Run of Heads

Write a function `flip_until_head_run` that takes an integer argument and keeps flipping a coin until it gets that many heads *in a row*.

For example:

```
> flip_until_head_run(2)
tails
heads
tails
heads
heads
```


Getting User Input

Another common use of `while` loops involves getting user input.

We can use the `input` function to ask the user to give us a number:

```
def prompt_num() :  
    return input("please enter a number: ")
```

but there's no guarantee they'll comply.

So we can keep asking until we get what we want:

```
def get_num() :  
    answer = ""  
    while not answer.isdigit() :  
        answer = input("please enter a number: ")  
    return int(answer)
```

Activity: Getting Your Way

Write a function called `gimme` that asks for a random item from a list, until the user gives in and agrees to give it to them.

For example:

```
> gimme()  
Can I have a balloon? PLEASE? no  
Can I have candy? PLEASE? No  
Can I have ice cream? PLEASE? no  
Can I have ice cream? PLEASE? I said, "no".  
Can I have ice cream? PLEASE? ohmygodwhy?  
Can I have french fries? PLEASE? no  
Can I have a pony? PLEASE? okay  
Yay! a pony!
```

To Do

Finish *Homework 6: More For-loops* on CodeRunner by 11:00AM next Thursday.

Finish on *Project 2 - What Does the Fox Say?* on CodeRunner by next Friday.